NAÏVE CODE

```
import numpy as np
import pandas as pd
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
# Dataset
messages = [
    'free win prize money',
    'free offer click now',
    'win free cash today',
    'hello how are you',
    'meeting tomorrow morning',
    'see you later today',
    'free cash win now click',
    'lunch tomorrow with team',
]
labels = ['spam','spam','spam','ham','ham','ham','spam','ham']


# Vectorize
```

```
vectorizer = CountVectorizer()

X = vectorizer.fit_transform(messages)

y = np.array(labels)

# Train/Test Split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Sklearn Naive Bayes

model = MultinomialNB()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("=== Sklearn Naive Bayes ===")

print("Actual   :", list(y_test))

print("Predicted:", list(y_pred))

print("Accuracy :", round(accuracy_score(y_test, y_pred), 4))

print(classification_report(y_test, y_pred))

# Probabilities

probs = model.predict_proba(X_test)

prob_df = pd.DataFrame(probs, columns=model.classes_)

prob_df['Predicted'] = y_pred

prob_df['Actual'] = list(y_test)

print(prob_df.round(4))

# Manual Naive Bayes from Scratch
```

```
class ManualNaiveBayes:
    def fit(self, X, y):
        self.classes = np.unique(y)
        self.priors = {}
        self.likelihoods = {}
        for c in self.classes:
            X_c = X[y == c]
            self.priors[c] = len(X_c) / len(X)
            self.likelihoods[c] = (X_c.sum(axis=0) + 1) / (X_c.sum() +
X.shape[1])
    def predict(self, X):
        preds = []
        for x in X:
            scores = {}
            for c in self.classes:
                score = np.log(self.priors[c])
                score += np.sum(np.log(self.likelihoods[c]) * x)
                scores[c] = score
            preds.append(max(scores, key=scores.get))
        return preds
X_dense = X.toarray()
```

```
X_train_d, X_test_d, y_train_d, y_test_d = train_test_split(X_dense, y,
test_size=0.3, random_state=42)

manual_nb = ManualNaiveBayes()

manual_nb.fit(X_train_d, y_train_d)

manual_preds = manual_nb.predict(X_test_d)

print("\n=== Manual vs Sklearn ===")

compare_df = pd.DataFrame({

    'Actual'    : list(y_test_d),

    'Manual NB' : manual_preds,

    'Sklearn NB': list(y_pred),

    'Match'     : [m == s for m, s in zip(manual_preds, list(y_pred))]

})

print(compare_df)

print("\nManual  Accuracy:", round(accuracy_score(y_test_d,
manual_preds), 4))

print("Sklearn Accuracy:", round(accuracy_score(y_test, y_pred), 4))
```